22AIE303 DBMS Labsheet:PL-SQL Lab1

Karthik R S | AM.SC.U4AIE23119

```
create table Emp(
    eno int primary key,
    ename varchar(20),
    sal numeric(10,2),
    dno int
);
create table Dept(
    dept_no int primary key,
    dname varchar(20),
    loc varchar(30)
);
create or replace function sum2(a1 int,b int)
    returns int as
$$
 Declare
    c int;
begin
    c:=a1+b;
    return c;
End;
$$
language 'plpgsql';

select sum2(5,10)
```

	sum2 integer
1	15

```
create or replace function odd(a1 int)
    returns varchar as
$$
begin
    if a1%2=0 then
        return 'even';
    else
        return 'odd';
    end if;
End;
$$
language 'plpgsql';

select odd(5)
```

	odd character varying
1	odd

```
create or replace function emp_no(empno Emp.eno%type)
    returns int as
$$
declare
    Salary Emp.sal%type;
begin
    select sal into Salary from Emp where eno=empno;
    return Salary;
End;
$$
language 'plpgsql';
```

```
create or replace function dept_no(depno Emp.dno%type)
    returns int as
$$
declare
    Sum1 int;
begin
    select count(eno) into Sum1 from emp where dno=depno;
    return Sum1;
End;
$$
language 'plpgsql';
```

```
create or replace function dep_name(depno Emp.dno%type)
    returns Dept.dname%type as
$$
declare
    deptname Dept.dname%type;
begin
    select dname into deptname from Dept,Emp where eno=empno and dno=dept_no;
    return deptname;
End;
$$
language 'plpgsql';
```

```
CREATE TABLE Emp1 (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(30),
    job_title VARCHAR(30),
    hire_date DATE DEFAULT CURRENT_DATE,
    salary NUMERIC(10,2),
    dept_no INT REFERENCES Dept(dept_no)
);
```

```sql
CREATE TABLE Product (
    prod_id INT PRIMARY KEY,
    prod_name VARCHAR(50) NOT NULL,
    price NUMERIC(10,2),
    category VARCHAR(30),
    stock_qty INT
);

CREATE TABLE Emp_Sales (
    sale_id SERIAL PRIMARY KEY,
    emp_id INT REFERENCES Emp1(emp_id),
    prod_id INT REFERENCES Product(prod_id),
    sale_date DATE,
    qty_sold INT
);
```

```sql
create or replace function check_commission_eligibility(p_emp_no emp.eno%type)
returns text as
$$
declare
    total_qty int;
begin
    select sum(qty_sold)
    into total_qty
    from emp_sales
    where emp_id = p_emp_no;

    if total_qty >= 1000 then
        return 'eligible for commission';
    else
        return 'not eligible for commission';
    end if;
```

```sql
    total_qty int;
begin
    select sum(qty_sold)
    into total_qty
    from emp_sales
    where emp_id = p_emp_no;

    if total_qty >= 1000 then
        return 'eligible for commission';
    else
        return 'not eligible for commission';
    end if;
end;
$$ language 'plpgsql';
```

2. Consider the following schema .
   Emp(eno, ename, sal)
   EMP_PROJ(eno, pno, prjt_hrs).

   a) Write a function ProjectLoad that returns the total project working hours for the given eno.
   b)
   c) Write a PL/SQL code to update the salary of an employee if the employee earn less than the average salary.

      New salary is current sal + difference between currrent sal and average salary
      .

```sql
create table emp_proj (
    eno int,
    pno int,
    prjt_hrs int,
    primary key (eno, pno)
);
create table item (
    item_no int primary key,
    item_name varchar(50),
    unit_price numeric(10,2)
);

create table transaction (
    tr_no serial primary key,
    item_no int references item(item_no),
    qty int
);

create table branches (
    br_no int primary key,
    br_name varchar(50),
    loc varchar(50)
);
```

```sql
create table customer (
    c_no int primary key,
    cname varchar(50),
    c_type char(1)
);

create table accounts (
    ac_no int primary key,
    br_no int references branches(br_no),
    cust_no int references customer(c_no),
    ac_type varchar(10),
    bal numeric(12,2)
);
```

```sql
CREATE OR REPLACE FUNCTION ProjectLoad(p_eno INT)
RETURNS INT AS
$$
DECLARE
    total_hrs INT;
BEGIN
    SELECT SUM(prjt_hrs)
    INTO total_hrs
    FROM EMP_PROJ
    WHERE eno = p_eno;

    IF total_hrs IS NULL THEN
        total_hrs := 0;
    END IF;


    WHERE eno = p_eno;

    IF total_hrs IS NULL THEN
        total_hrs := 0;
    END IF;

    RETURN total_hrs;
END;
$$ LANGUAGE plpgsql;

SELECT ProjectLoad(101);

CREATE OR REPLACE FUNCTION UpdateLowSalary()
RETURNS VOID AS
$$
DECLARE
    avg_sal NUMERIC;
BEGIN

    SELECT AVG(sal) INTO avg_sal FROM EMP;
    UPDATE EMP
    SET sal = sal + (avg_sal - sal)
    WHERE sal < avg_sal;
END;
$$ LANGUAGE plpgsql;
```

3. Consider the following tables
   Item(ino, iname, unit_price)
   Transaction(tr_no,ino,qty)

   Write a function to accept an item no from the user. If transaction has been made for less than 2 times for that item(check from transaction table), delete the item from the Item table.

```
CREATE OR REPLACE FUNCTION DeleteLowTransactionItem(p_item_no INT)
RETURNS TEXT AS
$$
DECLARE
    trans_count INT;
BEGIN
    SELECT COUNT(*) INTO trans_count
    FROM TRANSACTION
    WHERE item_no = p_item_no;

    IF trans_count < 2 THEN
        DELETE FROM ITEM WHERE item_no = p_item_no;
        RETURN 'Item deleted because it has less than 2 transactions.';
    ELSE
        RETURN 'Item not deleted – has 2 or more transactions.';
    END IF;
END;
```

4.

Consider a Bank database which includes the following tables.

ACCOUNTS(ac_no,br_no, cust_no,ac_type,bal)
BRANCHES(br_no, br_name, loc)
CUSTOMER(cno, cname, c_type)

a) Write a function that accepts a threshold value and a customer number . The program updates the c_type based on the threshold value. Ie. If balance> threshold then class A, else class B.
b) Write a function called CloseBranch that takes two arguments (the branch to be closed and the branch to take over the accounts) and transfers all accounts at the closing branch to the new branch and removes the closing branch.
c) Write a function that implements a "safe" withdrawal operation, that only permits a withdraw if there sufficient funds in the account to cover it.

```
CREATE OR REPLACE FUNCTION UpdateCustomerType(p_threshold NUMERIC, p_cust_no INT)
RETURNS TEXT AS
$$
DECLARE
    total_bal NUMERIC;
BEGIN
    SELECT SUM(bal) INTO total_bal
    FROM ACCOUNTS
    WHERE cust_no = p_cust_no;

    IF total_bal IS NULL THEN
        total_bal := 0;
    END IF;

    IF total_bal >= p_threshold THEN
        UPDATE CUSTOMER SET c_type = 'A' WHERE c_no = p_cust_no;
        RETURN 'Customer upgraded to Class A';
```

```
BEGIN
    SELECT SUM(bal) INTO total_bal
    FROM ACCOUNTS
    WHERE cust_no = p_cust_no;

    IF total_bal IS NULL THEN
        total_bal := 0;
    END IF;

    IF total_bal >= p_threshold THEN
        UPDATE CUSTOMER SET c_type = 'A' WHERE c_no = p_cust_no;
        RETURN 'Customer upgraded to Class A';
    ELSE
        UPDATE CUSTOMER SET c_type = 'B' WHERE c_no = p_cust_no;
        RETURN 'Customer set to Class B';
    END IF;
END;
```

```
CREATE OR REPLACE FUNCTION CloseBranch(p_old_br INT, p_new_br INT)
RETURNS TEXT AS
$$
BEGIN
    UPDATE ACCOUNTS
    SET br_no = p_new_br
    WHERE br_no = p_old_br;

    DELETE FROM BRANCHES WHERE br_no = p_old_br;

    RETURN 'Branch closed and accounts transferred successfully.';
END;
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE FUNCTION SafeWithdrawal(p_ac_no INT, p_amount NUMERIC)
RETURNS TEXT AS
$$
DECLARE
    current_bal NUMERIC;
BEGIN
    SELECT bal INTO current_bal FROM ACCOUNTS WHERE ac_no = p_ac_no;

    IF current_bal IS NULL THEN
        RETURN 'Account not found';
    END IF;

    IF current_bal >= p_amount THEN
        UPDATE ACCOUNTS
        SET bal = bal - p_amount
```

```
    IF current_bal IS NULL THEN
        RETURN 'Account not found';
    END IF;

    IF current_bal >= p_amount THEN
        UPDATE ACCOUNTS
        SET bal = bal - p_amount
        WHERE ac_no = p_ac_no;
        RETURN 'Withdrawal successful';
    ELSE
        RETURN 'Insufficient funds';
    END IF;
END;
$$ LANGUAGE plpgsql;
```